Name: Basinillo, Jewel Heart	**Date:** November 2, 2026
Course Code/Section: CPE 200-CPE21S2	**Professor Name:** Engr. Miko Gorospe

Final Practical Examination - Sorting Algorithms Using C++

Instructions

A teacher has recorded the examination scores of several students. The scores are currently arranged in random order, making it difficult to identify the highest and lowest scores.

Your task is to develop a C++ program that allows the teacher to arrange these scores using different sorting algorithms.

Requirements

1. Create a C++ program that accepts 10 examination scores from the user.
2. Store the scores in a one-dimensional array.
3. Display the original arrangement of the scores.
4. Create a menu that allows the user to select one of the following sorting algorithms:
    - 1 - Bubble Sort
    - 2 - Selection Sort
    - 3 - Insertion Sort
    - 4 - Shell Sort
    - 5 - Merge Sort
    - 6 - Quick Sort
    - 7 - Exit
5. Implement all six sorting algorithms using separate functions.
6. Allow the user to choose whether the scores will be arranged in ascending or descending order.
7. Display the sorted scores after executing the selected algorithm.
8. Allow the user to return to the main menu and select another sorting algorithm without restarting the program.
9. Ensure that the original scores are preserved so that each sorting algorithm operates on the same unsorted data.
10. Display an appropriate message when the user enters an invalid menu option.

Restriction

- Use C++ as the programming language.
- Use a one-dimensional array to store the examination scores.
- Use basic control structures such as if, else, for, while, and switch.
- Use a temporary variable when exchanging array elements.
- Do not use the built-in sort() or swap() functions.
- Do not use vectors or other STL containers.
- Each sorting algorithm must be implemented manually.
- Functions are required to organize the program.

PROGRAM CODE

```
#include<iostream>
using namespace std;

// BUBBLE SORT
void bubbleSort(int arr[], int order){
    int temp;
```

```
    for(int i = 0; i < 10-1; i++){
        for(int j = 0; j < 10-i-1; j++){

            if((order == 1 && arr[j] > arr[j+1]) ||
               (order == 2 && arr[j] < arr[j+1])){

                temp = arr[j];
                arr[j] = arr[j+1];
                arr[j+1] = temp;
            }
        }
    }
}

// SELECTION SORT
void selectionSort(int arr[], int order){
    int temp, selected;

    for(int i = 0; i < 10-1; i++){
        selected = i;

        for(int j = i+1; j < 10; j++){

            if((order == 1 && arr[j] < arr[selected]) ||
               (order == 2 && arr[j] > arr[selected])){
                selected = j;
            }
        }

        temp = arr[i];
        arr[i] = arr[selected];
        arr[selected] = temp;
    }
}

// INSERTION SORT
void insertionSort(int arr[], int order){
    int temp, j;

    for(int i = 1; i < 10; i++){
        temp = arr[i];
        j = i-1;

        while(j >= 0 && ((order == 1 && arr[j] > temp) ||
                         (order == 2 && arr[j] < temp))){
            arr[j+1] = arr[j];
            j--;
        }

        arr[j+1] = temp;
    }
```

```
}

// SHELL SORT
void shellSort(int arr[], int order){
    int temp;

    for(int gap = 10/2; gap > 0; gap /= 2){

        for(int i = gap; i < 10; i++){
            temp = arr[i];
            int j;

            for(j = i; j >= gap &&
                ((order == 1 && arr[j-gap] > temp) ||
                 (order == 2 && arr[j-gap] < temp)); j -= gap){

                arr[j] = arr[j-gap];
            }

            arr[j] = temp;
        }
    }
}

// MERGE SORT
void merge(int arr[], int left, int mid, int right, int order){
    int temp[10];
    int i = left;
    int j = mid+1;
    int k = left;

    while(i <= mid && j <= right){

        if((order == 1 && arr[i] <= arr[j]) ||
           (order == 2 && arr[i] >= arr[j])){

            temp[k] = arr[i];
            i++;
        }
        else{
            temp[k] = arr[j];
            j++;
        }
        k++;
    }

    while(i <= mid){
        temp[k] = arr[i];
        i++;
        k++;
    }
```

```
    while(j <= right){
        temp[k] = arr[j];
        j++;
        k++;
    }

    for(int x = left; x <= right; x++){
        arr[x] = temp[x];
    }
}

void mergeSort(int arr[], int left, int right, int order){

    if(left < right){
        int mid = (left+right)/2;

        mergeSort(arr, left, mid, order);
        mergeSort(arr, mid+1, right, order);

        merge(arr, left, mid, right, order);
    }
}

// QUICK SORT
int partition(int arr[], int low, int high, int order){
    int pivot = arr[high];
    int temp;
    int i = low-1;

    for(int j = low; j < high; j++){

        if((order == 1 && arr[j] <= pivot) ||
           (order == 2 && arr[j] >= pivot)){

            i++;

            temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }

    temp = arr[i+1];
    arr[i+1] = arr[high];
    arr[high] = temp;

    return i+1;
}

void quickSort(int arr[], int low, int high, int order){

    if(low < high){
```

```
        int pi = partition(arr, low, high, order);

        quickSort(arr, low, pi-1, order);
        quickSort(arr, pi+1, high, order);
    }
}


int main(){

    int score[10];
    int sorted[10];
    int menuchoice;
    int order;
    char again;

    cout << "STUDENT SCORE SORTING SYSTEM - BASINILLO\n";
    cout << "----------------------------------------\n";

    input:
    for(int i = 0; i < 10; i++){
        enter:
        cout << "Enter Examination Score " << i+1 << ": ";
        cin >> score[i];

        if(cin.fail()){
            cout << "- Please enter a valid number.\n\n";
            cin.clear();
            cin.ignore(10000,'\n');
            goto enter;
        }
    }

    cout << "----------------------------------------\n";
    cout << "Original Inputted Scores:\n";

    for(int i = 0; i < 10; i++){
        cout << score[i] << " ";
    }

    menu:
    cout << "\n----------------------------------------\n";
    cout << "SORTING ALGORITHMS\n";
    cout << "   1. Bubble Sort\n";
    cout << "   2. Selection Sort\n";
    cout << "   3. Insertion Sort\n";
    cout << "   4. Shell Sort\n";
    cout << "   5. Merge Sort\n";
    cout << "   6. Quick Sort\n";
    cout << "   7. Exit\n";
    cout << "----------------------------------------\n";
    cout << "Enter your Choice: ";
```

```
    cin >> menuchoice;

    if(cin.fail()){
        cout << "Invalid Choice. Please enter numbers 1 to 7 only.\n";
        cin.clear();
        cin.ignore(10000,'\n');
        goto menu;
    }

    cout << "\n-----------------------------------------\n";

    switch(menuchoice){

        case 1:
            cout << "Selected Algorithm: Bubble Sort\n";
            break;

        case 2:
            cout << "Selected Algorithm: Selection Sort\n";
            break;

        case 3:
            cout << "Selected Algorithm: Insertion Sort\n";
            break;

        case 4:
            cout << "Selected Algorithm: Shell Sort\n";
            break;

        case 5:
            cout << "Selected Algorithm: Merge Sort\n";
            break;

        case 6:
            cout << "Selected Algorithm: Quick Sort\n";
            break;

        case 7:
            cout << "Program is Exiting..\n";
            return 0;

        default:
            cout << "Invalid Choice. Please enter numbers 1 to 7 only..\n";
            goto menu;
    }

    ordermenu:
    cout << "\n-----------------------------------------\n";
    cout << "SELECT SORTING ORDER\n";
    cout << "   1. Ascending Order\n";
    cout << "   2. Descending Order\n";
    cout << "-----------------------------------------\n";
```

```
    cout << "Enter your Choice: ";
    cin >> order;

    if(cin.fail()){
        cout << "Please enter a valid number.\n";
        cin.clear();
        cin.ignore(10000,'\n');
        goto ordermenu;
    }

    if(order != 1 && order != 2){
        cout << "Invalid Choice. Please select 1 or 2 only.\n";
        goto ordermenu;
    }

    // COPY ORIGINAL SCORES TO SORTED ARRAY
    for(int i = 0; i < 10; i++){
        sorted[i] = score[i];
    }

    // EXECUTE SELECTED SORTING ALGORITHM
    switch(menuchoice){

        case 1:
            bubbleSort(sorted, order);
            break;

        case 2:
            selectionSort(sorted, order);
            break;

        case 3:
            insertionSort(sorted, order);
            break;

        case 4:
            shellSort(sorted, order);
            break;

        case 5:
            mergeSort(sorted, 0, 9, order);
            break;

        case 6:
            quickSort(sorted, 0, 9, order);
            break;
    }

    cout << "\n----------------------------------------\n";
    cout << "Sorted Scores:\n";

    for(int i = 0; i < 10; i++){
```

```
        cout << sorted[i] << " ";
    }

    cout << "\n----------------------------------------\n";

    againmenu:
    cout << "\nReturn to Main Menu? (Y/N): ";
    cin >> again;

    if(again == 'Y' || again == 'y'){
        goto menu;
    }
    else if(again == 'N' || again == 'n'){
        cout << "\nPlease enter the examination scores again.\n\n";
        goto input;
    }
    else{
        cout << "Invalid Choice. Please enter Y or N only.\n";
        goto againmenu;
    }

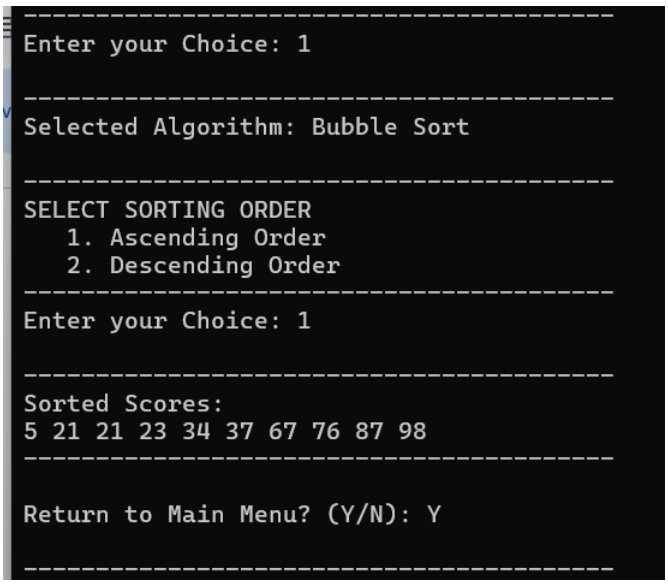
    return 0;
}
```

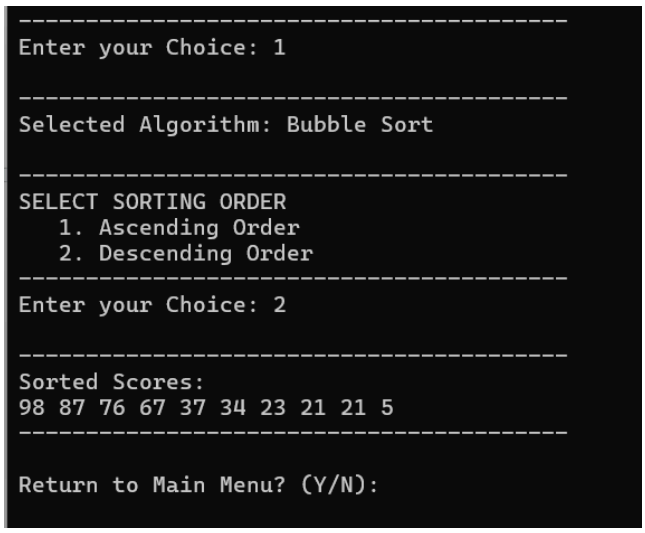
PROGRAM OUTPUT

1. **Enter Number and Menu**

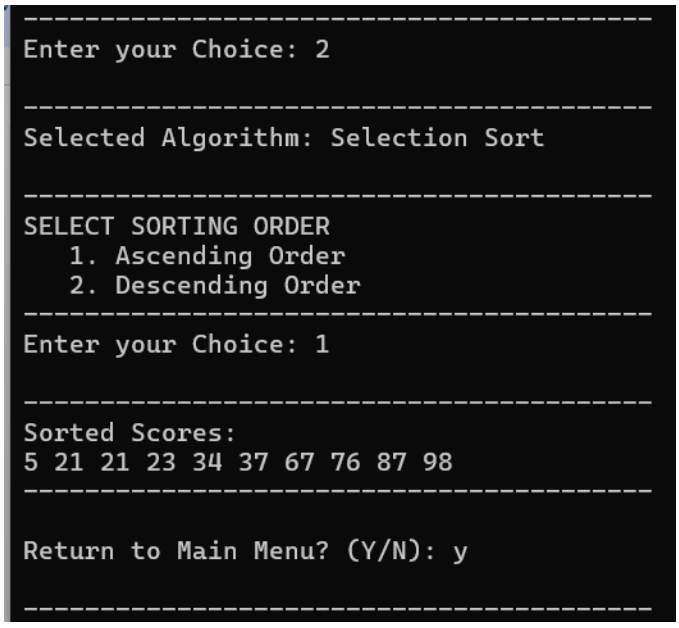
```
STUDENT SCORE SORTING SYSTEM - BASINILLO
----------------------------------------
Enter Examination Score 1: 23
Enter Examination Score 2: 21
Enter Examination Score 3: 67
Enter Examination Score 4: 76
Enter Examination Score 5: 34
Enter Examination Score 6: 98
Enter Examination Score 7: 21
Enter Examination Score 8: 37
Enter Examination Score 9: 87
Enter Examination Score 10: 5
----------------------------------------
Original Inputted Scores:
23 21 67 76 34 98 21 37 87 5
----------------------------------------
SORTING ALGORITHMS
   1. Bubble Sort
   2. Selection Sort
   3. Insertion Sort
   4. Shell Sort
   5. Merge Sort
   6. Quick Sort
   7. Exit
----------------------------------------
Enter your Choice:
```

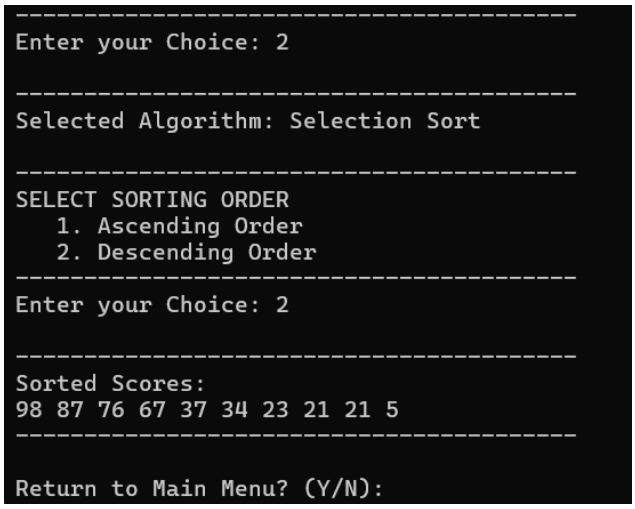
2. **Bubble Sort**



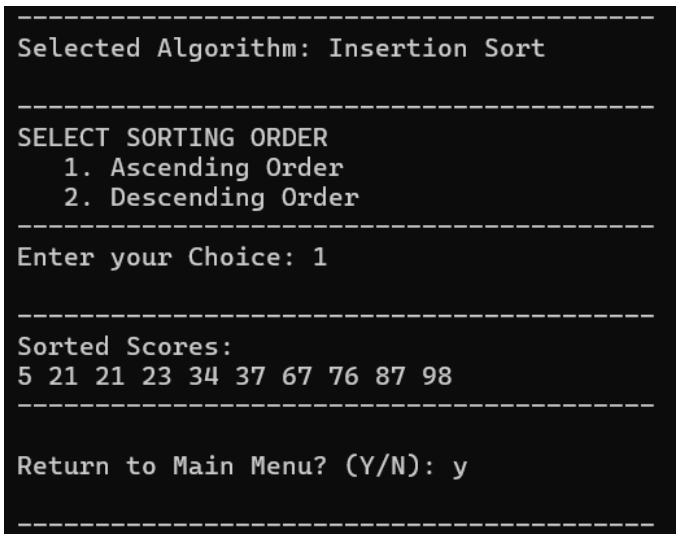


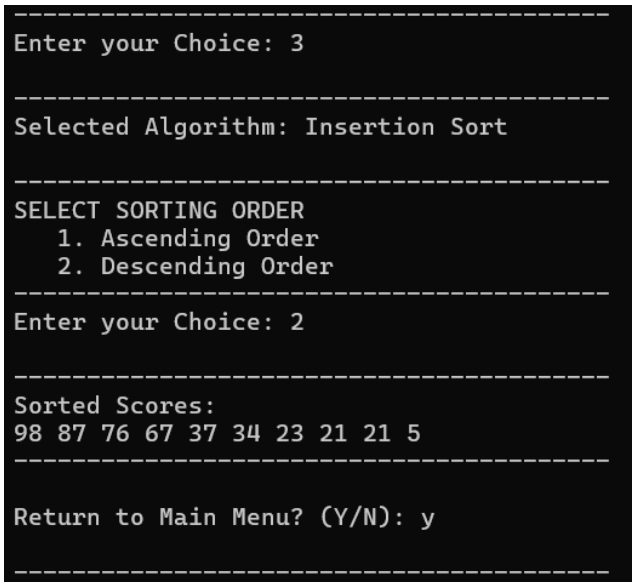
3. Selection Sort



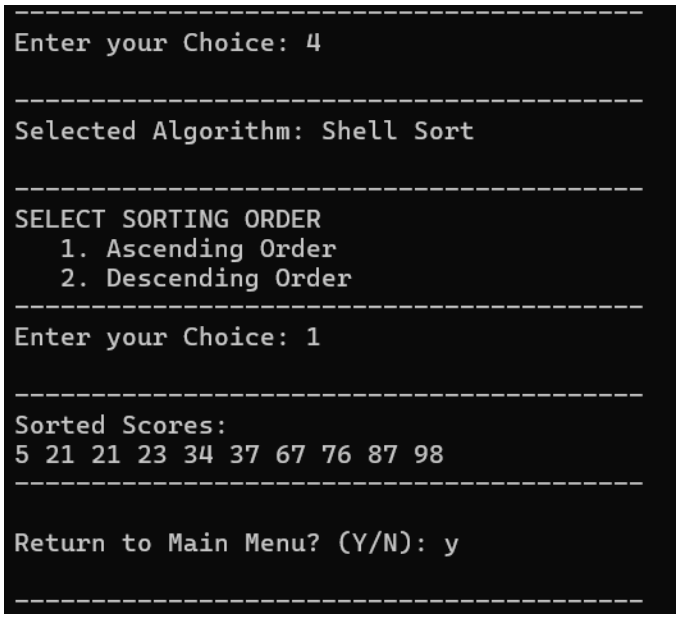


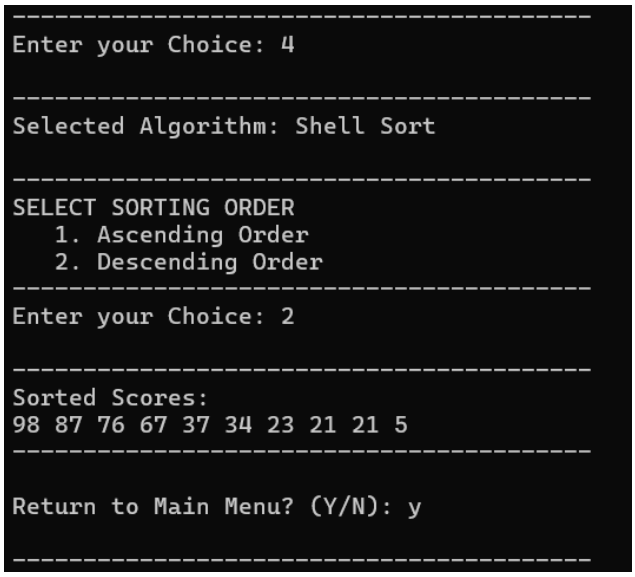
4. Insertion Sort



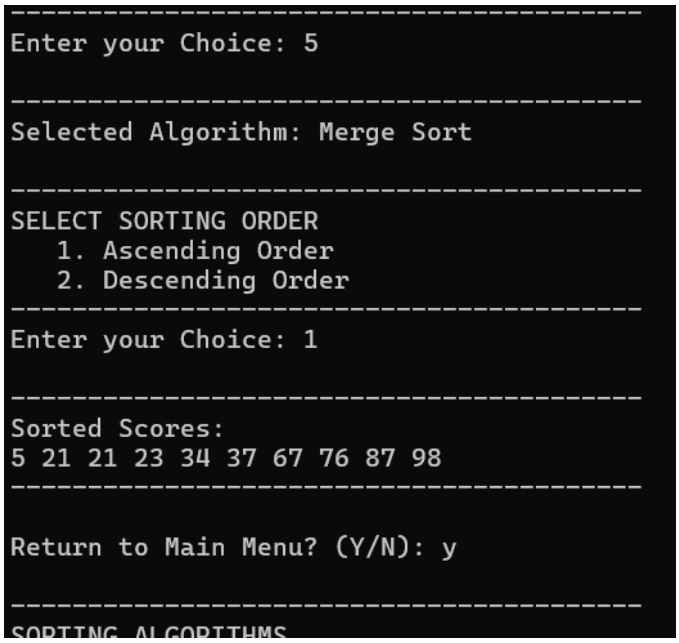


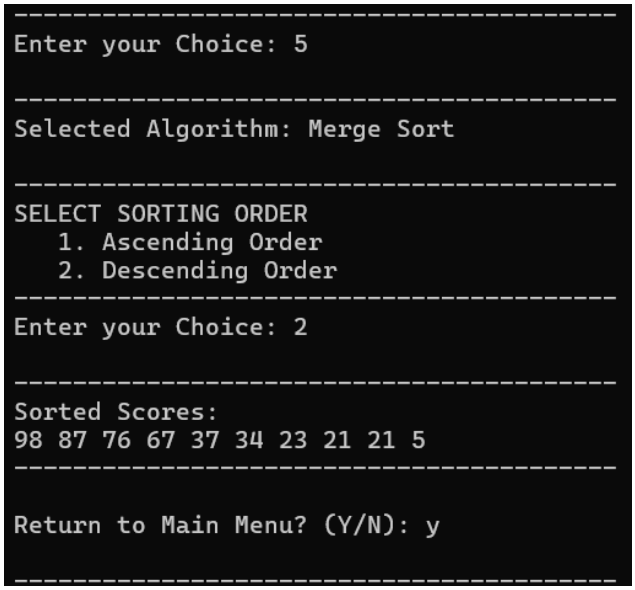
5. Shell Sort



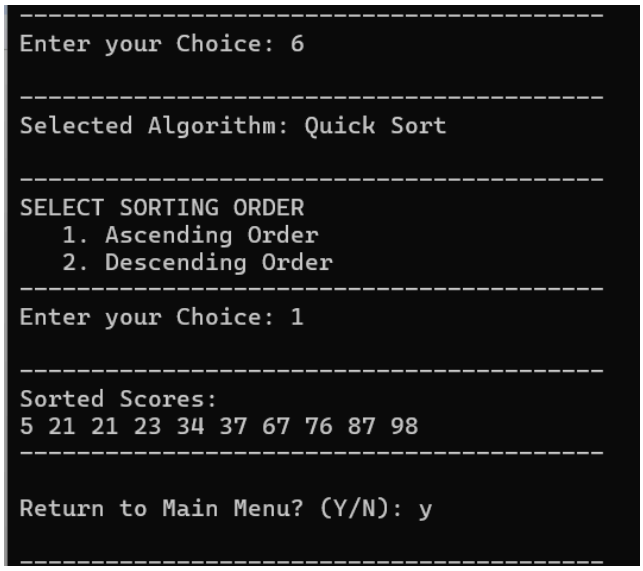


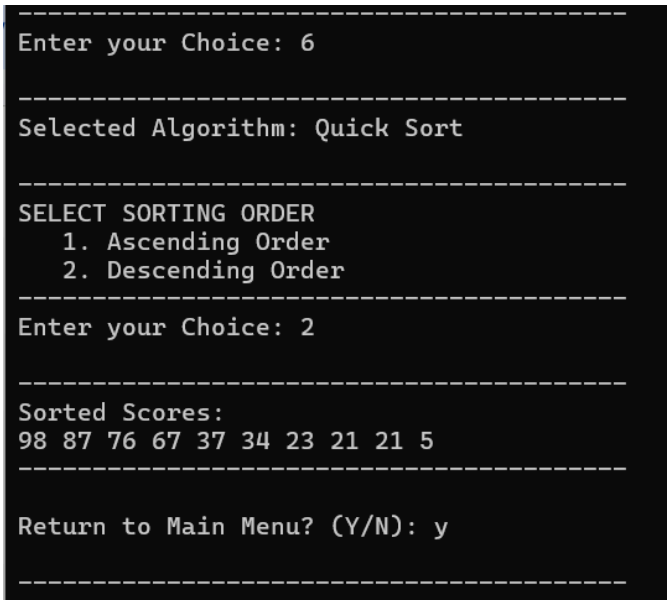
6. Merge Sort



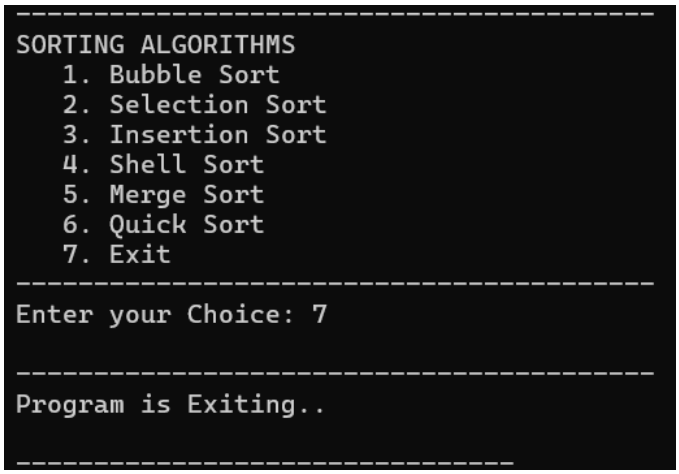


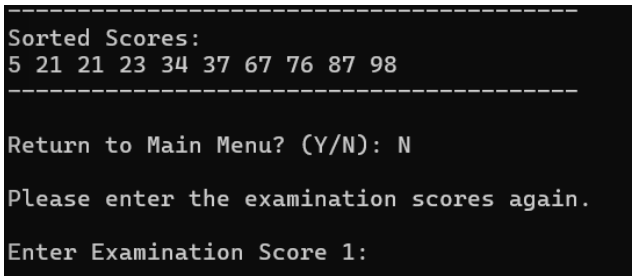
7. Quick Sort





8. Exit & N-Loop Back to Input





9. Error Handling

